

PROMPT FORMATS · TEMPLATE STRUCTURES

Styles of Prompt Formatting

A technical comparison of the three most common prompt template structures — Alpaca, ChatML, and plain Instruction format — and why getting the format exactly right matters as much as what you actually ask.

3

FORMATS

6

TEMPLATE EXAMPLES

1

CONVERSION WORKFLOW

9

REVISION CARDS

02 Topic Overview

Three template formats, plus the shared building blocks behind them.



Alpaca Format

Instruction / input / output triplets with markdown-style section headers.



ChatML Format

Role-tagged turns using special tokens for system, user, and assistant.



Instruction Format

The simplest single-turn instruction → response template, no roles.



Special Tokens

Reserved markers like `<|im_start|>` that delimit structure.



Role Structure

System / user / assistant separation used by multi-turn chat formats.



Template Mismatch

Using the wrong format for a model's fine-tuning silently degrades output.

03 Major Concepts

How each prompt format structures instructions and responses.

01 Alpaca Format

INSTRUCTION + INPUT

DEFINITION

The Alpaca format, popularized by Stanford's Alpaca dataset, structures each training example as three markdown-style sections: `### Instruction`, an optional `### Input` with extra context/data, and `### Response` for the expected output. It's a plain-text convention, not a set of reserved model tokens.

WHY IT MATTERS

Separating the instruction from its input data lets one instruction template generalize across many different inputs — e.g. the same "Summarize the following text" instruction reused with different article bodies. This format was widely adopted across early open-source instruction-tuned LLMs.

KEY CONCEPTS

- **Three-part structure** — Instruction, optional Input, Response.
- **Plain-text headers** — no reserved tokens, just markdown-like section titles.
- **Input is optional** — many examples use instruction + response only.
- **Single-turn** — designed for one instruction/response pair, not multi-turn chat.

EXAMPLE

```
### Instruction:
Summarize the following paragraph in one sentence.

### Input:
The transformer architecture processes sequences in parallel
using self-attention instead of recurrence, enabling much
faster training on large text corpora.

### Response:
Transformers use parallel self-attention instead of recurrence,
making large-scale training much faster.
```

💡ANALOGY

Alpaca format is like a fill-in-the-blank worksheet: a reusable instruction ("Summarize this"), a slot for the specific material to work on (the input), and a blank line for the answer (the response).

02 ChatML Format

ROLE-BASED · MULTI-TURN

DEFINITION

ChatML ("Chat Markup Language") wraps each conversation turn with explicit role tags and reserved **special tokens** — typically `<|im_start|>` and `<|im_end|>` — around a role name (system, user, or assistant) and its message content. These tokens are part of the tokenizer's vocabulary, not just visual text.

WHY IT MATTERS

Because the delimiters are true reserved tokens rather than plain text, the model can reliably tell where one speaker's turn ends and another begins — critical for multi-turn conversations, system-prompt persistence, and preventing user input from being mistaken for an assistant instruction.

KEY CONCEPTS

- **Reserved special tokens** — `im_start/im_end` are actual vocabulary entries, not just text.
- **Explicit roles** — system, user, and assistant turns are unambiguously separated.
- **Multi-turn native** — designed from the ground up for extended conversations.
- **Widely adopted** — the structural basis for many modern chat-tuned model APIs.

EXAMPLE

```
<|im_start|>system
You are a concise, helpful assistant.<|im_end|>
<|im_start|>user
What is the capital of Japan?<|im_end|>
<|im_start|>assistant
Tokyo.<|im_end|>
```

🔑KEY POINT

Because ChatML's role delimiters are real tokens in the model's vocabulary — not plain text a user could type — a well-trained model treats them as structurally trustworthy boundaries, which is part of why role-tagged formats are more robust against a user's message being confused for a system instruction.

⚠️COMMON MISTAKE

Typing `<|im_start|>` as literal text into a prompt for a model that doesn't use ChatML. If those characters aren't actually reserved tokens in that model's tokenizer, they get split into ordinary sub-word tokens and carry no special structural meaning at all.

03 Instruction Format

MINIMAL · SINGLE-TURN

DEFINITION

Plain instruction format is the simplest of the three: just an instruction section followed directly by a response section, with no separate input field and no role tags. It's often written as `### Instruction: ... ### Response:`, or sometimes even more minimally as a raw instruction string followed by a completion.

WHY IT MATTERS

It's the lowest-overhead format to construct training data or prompts in — useful when the task is fully self-contained in the instruction itself and doesn't need a separate data payload or conversational history. Many single-purpose fine-tuned models use this minimal structure.

KEY CONCEPTS

- **Two-part structure** — Instruction, then Response — no Input field.
- **No role tags** — unlike ChatML, there's no system/user/assistant separation.
- **Self-contained** — the instruction must carry all needed context on its own.
- **Foundation for Alpaca** — Alpaca format is essentially this, extended with an Input field.

EXAMPLE

```
### Instruction:
Write a professional out-of-office email reply for a
one-week vacation.

### Response:
Thank you for your email. I am currently out of office
with limited access to email and will respond upon my
return next week...
```

ANALOGY

Plain instruction format is like a sticky note with a single task written on it — no attachments, no back-and-forth, just "do this" and "here's the result." Compare that to Alpaca's worksheet-with-attached- material, or ChatML's full transcribed conversation with labeled speakers.

04 Comparison: Format at a Glance

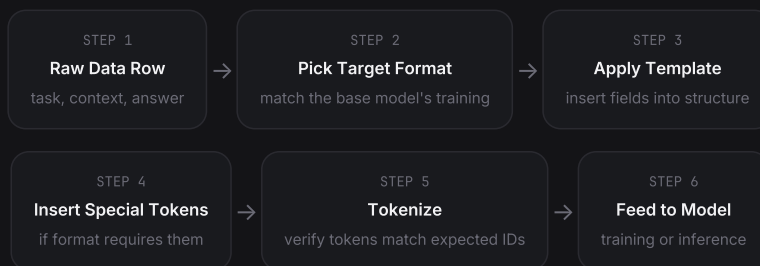
FORMAT	STRUCTURE	DELIMITERS	BEST FOR
Alpaca	Instruction + optional Input + Response	Plain-text markdown headers	Reusable instructions applied to varying data
ChatML	Role-tagged turns (system/user/assistant)	Reserved special tokens	Multi-turn conversation, chat APIs
Instruction	Instruction + Response only	Plain-text markdown headers	Simple, self-contained single-turn tasks

COMMON MISTAKE

Fine-tuning or prompting a model with a format different from what it was originally trained on. A model trained exclusively on ChatML-formatted conversations will often perform noticeably worse when prompted with Alpaca-style headers, since it never learned those text patterns as structural signals.

05 Workflow: Raw Data → Formatted Prompt

How a raw dataset row becomes a model-ready training example.



06 Step-by-Step: Formatting a Dataset Row

Converting the same raw example into all three formats.

1 Start from a raw record

A typical dataset row has fields like `instruction`, `input` (optional), and `output`.

2 Check the base model's expected format

Read the model card / tokenizer config — using the wrong template silently hurts output quality.

3 Render the template with a function

Use a small formatting function per template so the structure stays consistent across your whole dataset.

4 Verify special tokens tokenize correctly

For ChatML, confirm `<|im_start|>` maps to a single reserved token ID, not a split sequence of ordinary sub-words.

PYTHON · FORMAT THE SAME ROW THREE WAYS

```
row = {
    "instruction": "Translate to French.",
    "input": "Good morning!",
    "output": "Bonjour !",
}

def to_alpaca(r):
    return (
        f"### Instruction:\n{r['instruction']}\n\n"
        f"### Input:\n{r['input']}\n\n"
        f"### Response:\n{r['output']}"
    )

def to_chatml(r):
    return (
        "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
        f"<|im_start|>user\n{r['instruction']} {r['input']}<|im_end|>\n"
    )
```

```
f"<|im_start|>assistant\n{r['output']}<|im_end|>"
)

def to_instruction(r):
    combined = f"{r['instruction']} {r['input']}".strip()
    return f"### Instruction:\n{combined}\n\n### Response:\n{r['output']}"
```

⚡KEY POINT

Whichever format you choose, apply it **identically** at both training/fine-tuning time and inference time. A mismatch between how a model was trained and how it's later prompted is one of the most common causes of degraded output quality after fine-tuning.

07

Final Summary

What to remember

✓ Alpaca separates a reusable instruction from a variable input, both feeding one response.	✓ ChatML uses real reserved tokens to tag system/user/assistant turns for multi-turn chat.
✓ Instruction format is the minimal instruction → response pair, no input or roles.	✓ Alpaca is effectively Instruction format + an Input field .
✓ ChatML's delimiters are true vocabulary tokens , not plain text — that's what makes them reliable.	✓ Always match the format used at training time when prompting at inference time.

08

Quick Revision Cards

📄 Alpaca ➤ Instruction + optional Input + Response ➤ Plain-text markdown headers ➤ Great for reusable instructions	💬 ChatML ➤ < im_start > / < im_end > reserved tokens ➤ Explicit system/user/assistant roles ➤ Built for multi-turn conversation	📋 Instruction ➤ Instruction + Response only ➤ No roles, no input field ➤ Simplest, most minimal format
🔑 Special Tokens ➤ Reserved vocabulary entries ➤ Not the same as literal text ➤ Must match tokenizer config exactly	👤 Roles ➤ System sets standing behavior ➤ User / assistant alternate turns ➤ Core to ChatML, absent from Alpaca	⚠ Format Mismatch ➤ Wrong format silently hurts quality ➤ Check the model card before prompting ➤ Train and infer with the same template

09

Glossary / Terminology Cheat-Sheet

Prompt Template A reusable text structure with placeholders filled in per example.	Special Token A reserved vocabulary entry (like < im_end >) with structural, not literal, meaning.	System Prompt A standing instruction set at the start of a conversation that shapes overall behavior.
Multi-Turn A conversation format spanning several back-and-forth exchanges, not just one Q&A pair.	Fine-Tuning Data Formatted instruction/response examples used to further train a base model.	Template Mismatch Prompting a model with a different format than it was trained on, degrading quality.
Model Card Documentation describing a model's training details, including its expected prompt format.	Tokenizer Config The file defining a model's vocabulary, including any reserved special tokens.	Chat Template Model-specific logic (often a Jinja template) that renders role-tagged messages into final text.